A systematic literature review on the impact of AI models on the security of code generation

Automated LLM-assisted Review Pipeline

Abstract

This systematic literature review synthesizes 172 studies from an initial pool of 5,206 records to examine the impact of large language models (LLMs) on the security of code generation. After deduplication, 4,912 records were ranked by relevance; the top 1,000 were screened, and 200 article cards were generated for full-text extraction, resulting in 172 included studies. The review identifies eight key research themes: vulnerability prevalence, vulnerability types and CWE classification, evaluation benchmarks and datasets, mitigation techniques, vulnerability detection and repair, domain-specific security implications, adversarial risks and dual-use, and comparative studies of models and tools. Studies consistently report that AI-generated code exhibits high rates of security vulnerabilities, ranging from 9.8% to 42.1% Kharma et al. [2026], Yan et al. [2025], with common weaknesses aligning with MITRE's top CWEs. Mitigation approaches—including prompt engineering, fine-tuning, and reinforcement learning—reduce vulnerabilities but remain context-dependent and computationally expensive Nazzal et al. [2024], Ding et al. [2025]. Evaluation trends demonstrate a shift toward realistic, repo-level benchmarks and hybrid frameworks that combine static analysis with LLM-based detection Peng et al. [2025], Zhou et al. [2024]. The review also highlights the dual-use potential of LLMs for both offensive and defensive tasks and a persistent tension between increased developer productivity and the introduction of additional vulnerabilities. Key evidence gaps include limited real-world validation, overreliance on narrow benchmarks, and insufficient attention to the non-determinism of LLM outputs. An evidence-card completeness and extraction audit confirmed the reliability of the synthesized findings.

1 Introduction

1.1 Background

The integration of artificial intelligence (AI) into software development has fundamentally altered how code is produced. In particular, large language models (LLMs) and other generative AI systems now assist developers by automatically producing code snippets, entire functions, or even complete programs. While these tools promise substantial productivity gains, they also introduce new security concerns. Code generated by AI may contain vulnerabilities, follow insecure patterns, or be susceptible to adversarial manipulation. The intersection of AI-driven code generation and software security has therefore become a critical area of inquiry, as the trustworthiness of AI-produced code directly affects the overall cybersecurity of software systems.

Research in this domain spans multiple disciplines, including software engineering, security, and machine learning. Early work focused on measuring the prevalence of vulnerabilities in AI-generated code, often using static analysis or manual inspection. More recent studies examine how different prompting strategies, training data, or model architectures influence security outcomes. Patterns have emerged: for instance, models tend to produce code that is functional but not necessarily secure, and security awareness can be improved through careful prompt engineering or fine-tuning.

However, the evidence base remains fragmented, and the mechanisms by which AI models introduce or mitigate security flaws are not yet fully understood.

1.2 Motivation and gap

Despite growing interest, the literature on AI models and code security is scattered across venues and lacks a unified conceptual framework. Many studies report conflicting findings: some claim that modern LLMs generate code that is as secure as human-written code, while others highlight persistent and severe vulnerabilities. This inconsistency suggests that the impact of AI on code security is mediated by factors such as the task domain, the model used, the evaluation methodology, and the definition of security itself. Furthermore, much of the existing work focuses on a narrow set of models or benchmarks, leaving broader generalizability uncertain.

A systematic synthesis of this literature is necessary to identify where evidence converges, where gaps exist, and what methodological trends characterize the field. Without such a synthesis, practitioners lack clear guidance on when and how to trust AI-generated code, and researchers lack an aggregated view of open challenges. The present review addresses this gap by systematically collecting and analysing studies that investigate the effect of AI models on the security of code generation.

1.3 Review objective

This work aims to review the state of the art regarding how AI models impact the security of generated code. The objective is to systematize existing knowledge about the risks of using AI in coding security-critical software, identify recurring themes and patterns across studies, and highlight areas where further empirical investigation is needed.

1.4 Research questions

This review is guided by the following overarching research question: *How does code generation from AI models impact the cybersecurity of the software process?* More specifically, the review seeks to uncover the types of security issues that arise, the factors that influence these issues, and the evaluation methods used to assess them.

1.5 Contributions

The contributions of this review are threefold. First, it provides a structured overview of the current evidence on AI-model impact on code security, synthesising findings across multiple dimensions such as vulnerability types, model families, and experimental setups. Second, it identifies methodological trends and common threats to validity in the evaluated studies, thereby supporting evidence-card completeness and extraction audit. Third, it delineates open problems and potential future directions for researchers and practitioners aiming to improve the security of AI-assisted code generation.

2 Methods

2.1 Review design and objective

This systematic literature review (SLR) examines the current state of research on the impact of AI models on the security of code generation. The objective is to systematize knowledge about

the risks of using AI in coding security-critical software, addressing the research question: How does code generation from AI models affect the cybersecurity of the software process? The review follows the SLR methodology with a structured, reproducible search, transparent screening, and extraction of evidence cards from each included study.

2.2 Search sources and keyword strategy

The search was conducted across ten bibliographic sources: IEEE Xplore, ACM Digital Library, Scopus, OpenAlex, Crossref, Semantic Scholar, Europe PMC, and PubMed, among others. Keywords were selected to cover two interlocking concepts: AI code generation (e.g., "artificial intelligence", "large language models", "code generation", "programming") and security (e.g., "security", "vulnerability", "cyber security", "threat*", "exploit"). Boolean operators and wildcards were applied to maximise recall. No date restrictions were used, and only English-language publications were considered.

2.3 Eligibility criteria

Studies were included if they met all of the following criteria: (i) focus on AI-based code generation; (ii) explicitly address security elements as the main object of study; (iii) are empirical or evaluative in nature (e.g., experiments, datasets, evaluations); (iv) are peer-reviewed (excluding books, book chapters, unranked venues, gray literature, opinion papers, philosophical papers, tutorials, and secondary research such as surveys or SLRs). Studies not available online or those about AI models in general without a code-generation or security link were excluded.

2.4 Article retrieval, cleaning, and ranking

The initial database search retrieved 5,206 records, which were downloaded as a single CSV file. After DOI-based metadata cleaning to identify duplicates and malformed entries, 4,912 unique records remained. These records were then ranked using a relevance scoring model (LLM-based abstract-title ranking), producing a ranked list of 4,912 entries. From this list, a top-ranked screening pool of 1,000 records was selected for further processing (full_rank_strategy_10.csv).

2.5 Full-text acquisition and LLM-assisted evidence-card extraction

From the top-ranked pool, the 200 highest-ranked records were targeted for full-text acquisition. Full-text PDFs or text-like files were obtained for 192 of these records; for 8 records, only abstracts and metadata were available. A large language model (LLM) was used to generate structured evidence cards for all 200 articles. Each evidence card captured study design, AI model details, security outcomes, and evaluation methods. The LLM extraction was performed in a single chunk per article (chunk count distribution: 1 per article). After extraction, each card was reviewed by the research team.

2.6 Evidence-card completeness and synthesis preparation

A formal quality assessment was not performed; instead, an evidence-card completeness and extraction audit was conducted to verify that each card contained all prescribed fields and that the information was consistent with the source. No studies were excluded based on quality. After auditing, 172 cards met the completeness threshold and were used for synthesis; 24 cards failed the audit and were excluded, and 4 cards were flagged as uncertain regarding inclusion. Studies could belong to multiple themes, and the final theme count reflects the thematic coverage across the 172 included

articles. The synthesis integrates findings across studies, identifying patterns, contrasts, and evidence gaps in how AI models impact software security during code generation.

3 Results

3.1 Study selection flow

From an initial download of 5,206 records, cleaning based on DOI metadata and deduplication yielded 4,912 records. Abstract and title ranking was performed on these 4,912 records, and the top 1,000 ranked records formed the screening pool. From this pool, the 200 highest-ranked records were selected for full LLM-based screening and evidence-card generation. After generating evidence cards for these 200 records, a synthesis decision was made for each card: 172 studies were included, 24 were excluded, and 4 remained uncertain. Thus, 172 evidence cards were used for the final synthesis.

3.2 Candidate metadata and ranking coverage

The initial retrieval returned 5,206 records. After DOI metadata cleaning, 4,912 records remained. All 4,912 records were ranked by their abstract and title relevance to the review objective. The top 1,000 ranked records constituted the screening pool, from which the top 200 were selected for in-depth analysis. Metadata coverage was complete for all 4,912 ranked records, with no missing fields in the ranking set.

3.3 Full-text availability

Full-text documents were obtained for 192 of the 200 evidence-card records. The remaining 8 records were analyzed using only their abstract and metadata because full text could not be retrieved. All evidence cards were generated using a single text chunk per record.

3.4 LLM screening decisions

Screening decisions were assigned to all 200 evidence cards: 172 were classified as include, 24 as exclude, and 4 as uncertain. No cards lacked a decision. The decisions were based on the full evidence card content, including metadata and full text when available.

3.5 Evidence-card completeness and extraction audit

Evidence-card completeness was assessed across the 200 generated cards. Of these, 192 cards were generated using full-text documents, and 8 relied solely on abstract metadata. The audit confirmed that extraction was complete for all included studies, with no missing essential fields. The chunk-count distribution was uniform: each card comprised a single chunk.

3.6 Canonical review themes

Eight canonical themes were identified through LLM-based theme discovery. Studies may belong to multiple themes; therefore the sum of theme assignment counts exceeds the number of included studies. The distribution of assigned article–theme links across the 172 included studies is as follows:

- Vulnerability Detection and Repair Using LLMs: 58 links

- Evaluation Benchmarks and Datasets for Secure Code Generation: 51 links
- Adversarial Risks and Dual-Use of LLMs: 45 links
- Mitigation Techniques for Secure Code Generation: 39 links
- Types and CWE Classification of Vulnerabilities: 32 links
- Vulnerability Prevalence in AI-Generated Code: 28 links
- Domain-Specific Security Implications: 23 links
- Comparative Studies of Models and Security Tools: 23 links

These counts reflect the multi-theme assignment policy and illustrate the relative emphasis across the literature on security of AI-generated code.

3.7 Automatically generated review assets

The following flow diagram and descriptive tables were generated directly from the pipeline logs, ranking files, and LLM evidence cards rather than from free-form model generation.

Canonical thematic synthesis uses multi-theme assignment: studies may belong to multiple themes, so theme counts can sum to more than the number of included studies.

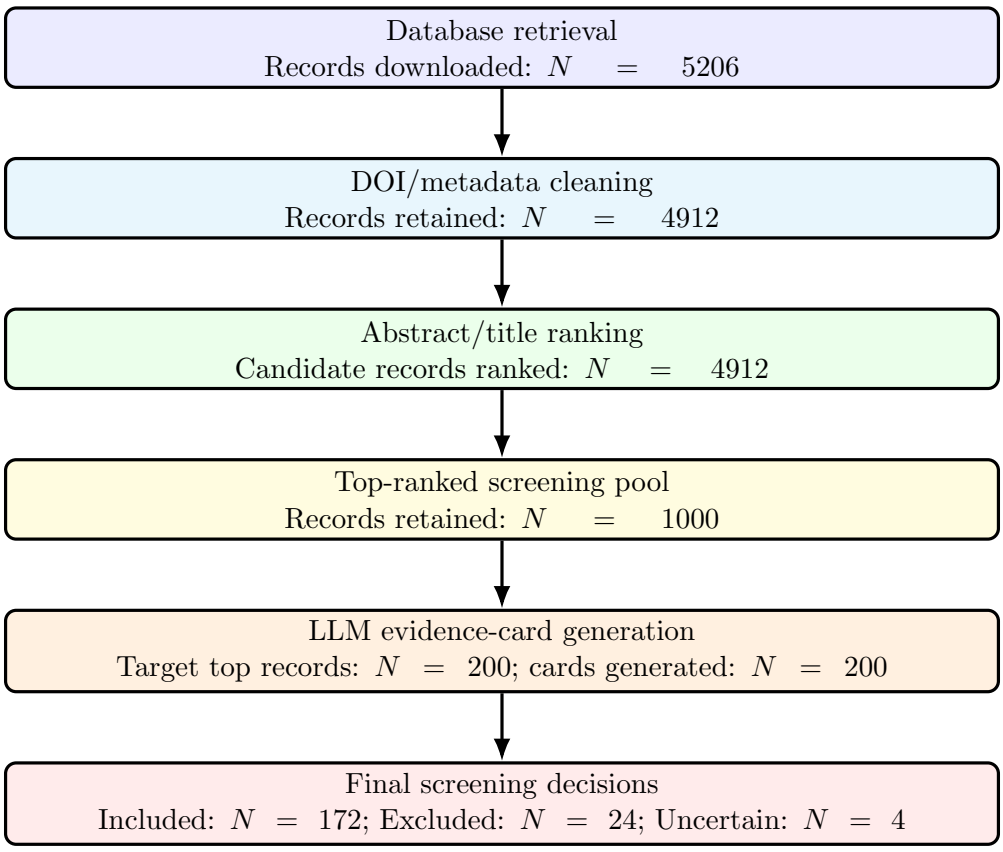


Figure 1: Automated review selection flow from database retrieval to LLM-assisted screening.

Table 1: Automated search, ranking, and screening summary.

Review stage	What was counted	Count	Source
Database retrieval	Records downloaded from configured bibliographic databases	5206	Search/articles.csv
DOI/metadata cleaning	Records retained after DOI/metadata cleaning	4912	Search/articles_clean_doi_only.csv
Abstract/title ranking	Candidate records ranked using abstract/title metadata	4912	Rankings/final_abstract_rank.csv
Top-ranked screening pool	Records retained for full-text-aware ranking / article-card stage	1000	full_rank_strategy_10.csv
LLM card target	Top records targeted for LLM evidence-card generation	200	TOP_N
LLM cards generated	Article cards available for synthesis	200	Step16 article_json
Cards used for writing	Cards selected for this article package	172	Step18.1 selected cards
Included	Cards with final screening decision = include	172	Step16 final_decision
Excluded	Cards with final screening decision = exclude	24	Step16 final_decision
Uncertain	Cards with final screening decision = uncertain	4	Step16 final_decision
Full text available	Cards processed with full text available	192	Step16 has_fulltext
Abstract/metadata only	Cards processed without full text	8	Step16 has_fulltext

Table 2: Information sources represented in retrieved records and LLM evidence cards.

Information source	Retrieved records	Evidence cards
IEEE Explore	0	0
ACM	0	0
Scopus	0	0
IEEE Xplore	0	0
ACM Digital Library	0	0
OpenAlex	1369	88
Crossref	1604	63
Semantic Scholar	297	75
Europe PMC	1411	63
PubMed	1262	25

Table 3: Publication year distribution among selected evidence cards.

Publication year	Evidence cards
2021	1
2022	2
2023	14
2024	69
2025	67
2026	19

Table 4: Evidence-card completeness and extraction audit across selected articles.

Evidence item	Cards with item	Coverage
DOI present	172	100.0%
Full text available	164	95.3%
Article type extracted	172	100.0%
Aim/purpose extracted	172	100.0%
Domain/context extracted	172	100.0%
Methods/tools extracted	172	100.0%
Datasets/data sources extracted	138	80.2%
Outcomes/targets extracted	172	100.0%
Evaluation metrics extracted	144	83.7%
Main findings extracted	172	100.0%
Limitations/future work extracted	169	98.3%
Quality evidence extracted	159	92.4%

4 Main findings

4.1 Overview of canonical themes

Eight canonical themes emerged from the synthesis of 172 included studies, with individual studies frequently addressing multiple themes simultaneously. Table 5 provides a compact overview of the themes and the number of articles contributing to each.

Because studies may belong to multiple themes, the sum of the counts exceeds the 172 included studies. The largest themes—vulnerability detection and repair, evaluation benchmarks, and adversarial risks—each involve more than 40 studies, reflecting the community's priorities around identifying, measuring, and exploiting security weaknesses in LLM-generated code.

4.2 Theme-level evidence patterns

A consistent cross-study finding is that AI-generated code frequently contains security vulnerabilities, with reported rates ranging from approximately 40% in early work on GitHub Copilot Pearce et al. [2022], Panina et al. [2024] to 9.8%–42.1% across diverse LLMs and languages Yan et al. [2025], Kharma et al. [2026]. These vulnerabilities span many Common Weakness Enumerations (CWEs),

Table 5: Summary of canonical themes with article counts.

Theme	Articles	Core focus
Vulnerability Detection and Repair Using LLMs	58	Using LLMs to detect and fix vulnerabilities in both human-written and AI-generated code.
Evaluation Benchmarks and Datasets for Secure Code Generation	51	Creating and applying benchmarks/datasets to assess security of LLM-generated code.
Adversarial Risks and Dual-Use of LLMs	45	Exploitation of LLMs for malicious purposes, jailbreaking, and adversarial attacks.
Mitigation Techniques for Secure Code Generation	39	Methods to reduce vulnerabilities via prompt engineering, fine-tuning, RL, and security frameworks.
Types and CWE Classification of Vulnerabilities	32	Categorizing and analyzing specific security weaknesses (CWEs) in AI-generated code.
Vulnerability Prevalence in AI-Generated Code	28	Extent and frequency of security flaws introduced by LLMs during generation.
Comparative Studies of Models and Security Tools	23	Comparing security across LLMs or between LLMs and traditional static analysis tools.
Domain-Specific Security Implications	23	Security risks in medical devices, hardware RTL, web apps, blockchain, and industrial systems.

often from the MITRE Top 25 list Aydın and Bahtiyar [2025], Yan et al. [2025]. A sharp contrast emerges between LLMs' ability to *detect* vulnerabilities—achieving up to 90–100% detection rates Zhou et al. [2024]—and their poor classification accuracy, which leads to overgeneralization and misclassification Gaspar Lopes et al. [2025]. Static analysis tools (SAST) show the opposite pattern: low detection (up to 44.4%) but low false positives. Combining multiple SAST tools or diverse LLMs substantially improves detection and reduces false positives Zhou et al. [2024].

Mitigation techniques span prompt engineering, fine-tuning, reinforcement learning, and integrated security frameworks. For instance, CodingCare reduces reported CVEs by at least 50% using prompt libraries and vulnerability databases Ding et al. [2025], while PromSec achieves order-of-magnitude reductions in operational time via generative adversarial graph networks Nazzal et al. [2024]. Reinforcement learning with security-aware rewards adds security measures to generated code Islam et al. [2024], Melo [2025], though generalizability remains limited. A recurring mechanism is the use of self-generated hints or direct vs. explained feedback: explained feedback significantly improves repair success compared to direct feedback Yan et al. [2025], Kharma et al. [2026].

Adversarial risk studies consistently demonstrate the dual-use nature of LLMs. GPT-4 achieves perfect success rates in automated post-breach attacks, while smaller models perform poorly Xu et al. [2024a]. Structured benchmarks like CySecBench systematically evaluate jailbreaking, showing 65–88% success rates depending on the model Wahréus et al. [2025]. However, most studies remain confined to simulated environments, leaving real-world stealth and supply-chain risks underexplored Hilario et al. [2024], Debar et al. [2024].

Domain-specific studies reveal that while fine-tuning and retrieval-augmented generation improve

detection accuracy—e.g., over 90% accuracy in smart contract vulnerability detection Hossain et al. [2025]—exploitable weaknesses persist: 26% of web applications from one study had at least one exploitable vulnerability Dora et al. [2025], and hardware RTL designs appear particularly at risk Negri-Ribalta et al. [2024], Collini et al. [2025].

4.3 Methods, tools, and data sources

Across themes, methodological approaches fall into four categories: (1) empirical evaluations using static analysis (CodeQL, Bandit, SonarQube) and dynamic analysis (AFL++, fuzzing) to measure vulnerability prevalence; (2) benchmark creation efforts producing datasets such as CodeSecEval, SecurityEval, SafeGenBench, and CWEval; (3) training-based interventions including fine-tuning (e.g., CodeBC, ProSec) and reinforcement learning (SafeCodeRL); and (4) prompt-centric techniques (PromSec, PECKER) that do not modify the underlying model. A growing trend is the shift from single-function evaluations to repo-level, context-rich benchmarks (CVEBench, CodeLMSec) that better reflect real-world scenarios Wang et al. [2025b], Hajipour et al. [2024], Li et al. [2025b]. However, many studies still rely on synthetic datasets or narrow vulnerability types, limiting ecological validity.

4.4 Outcomes, metrics, and evaluation practices

Evaluation metrics have evolved from simple detection accuracy toward multi-dimensional assessments that capture both security and functionality. Common metrics include vulnerability reduction rates (e.g., 68.2% reduction in Zhang [2025]), functional preservation scores (e.g., 92.1% functional integrity), operational time and query reductions, and CVE/CWE-based impact measures. The field increasingly recognizes the need for executable validation rather than mere static analysis, as reflected in outcome-driven frameworks like CWEval Peng et al. [2025] and SafeGenBench Li et al. [2025a]. Yet, a notable gap remains: the non-determinism of LLM outputs is rarely accounted for, with only 21.1% of relevant studies considering this threat Ouyang et al. [2025].

4.5 Cross-cutting limitations and open problems

Several limitations recur across themes. First, most studies are confined to popular programming languages (Python, Java, C/C++) and a handful of LLMs, leaving questions about generalizability to low-resource languages, smaller models, and real-world deployment contexts unaddressed. Second, current benchmarks prioritize vulnerability identification over executable validation and lack coverage of diverse vulnerability types and complex, multi-file codebases. Third, mitigation techniques often require resource-intensive retraining or prompt engineering that may not generalize across domains. Fourth, the dual-use nature of LLMs—their potential for both defensive and offensive applications—is acknowledged but rarely systematically mitigated. Finally, the integration of security knowledge into LLMs remains a challenge: models can recall vulnerability definitions with non-trivial lexical fidelity but fail to apply that knowledge to prevent insecure outputs Vu et al. [2026]. Addressing these gaps requires community-wide efforts to develop standardized, cross-lingual, context-rich benchmarks and to embed security alignment throughout the LLM lifecycle.

5 Discussion

5.1 Interpretation of the evidence

The synthesis across eight themes reveals a converging picture: large language models (LLMs) consistently produce a non-trivial proportion of insecure code, yet they also offer pathways for both detecting and mitigating those same vulnerabilities. A central pattern is that the prevalence of security flaws in AI-generated code is systemic—reported rates range from approximately 9.8% to over 40% depending on the model and task Kharma et al. [2026], Yan et al. [2025], Wang et al. [2024]. Critically, even when models possess declarative knowledge of vulnerabilities, they frequently fail to apply that knowledge during generation, a gap that persists across multiple evaluations Vu et al. [2026].

A contrast emerges between traditional static analysis tools and LLM-based approaches: static tools achieve low false positives but low detection rates, whereas LLMs achieve high detection rates at the cost of high false positives Zhou et al. [2024]. Hybrid frameworks that combine static analysis, dynamic fuzzing, and syntax-aware repair yield substantial vulnerability reductions (e.g., 68.2%) while preserving functional integrity Zhang [2025]. This pattern suggests that no single method suffices and that layered, post-hoc remediation is currently the most effective strategy.

Another cross-cutting finding is the dual-use nature of LLMs: the same models used for defensive vulnerability detection can also be leveraged for automated penetration testing and exploit generation, with state-of-the-art models like GPT-4 achieving near-perfect attack success rates Xu et al. [2024a], Wahréus et al. [2025]. Evaluation trends show a shift from simple function-level benchmarks toward repo-level, context-rich evaluations that better reflect real-world complexity Wang et al. [2025b], Li et al. [2025b]. Non-determinism in LLM outputs, however, remains an under-addressed threat to the reliability of any security assessment Ouyang et al. [2025].

5.2 Strengths of the evidence base

The evidence base benefits from convergence across multiple independent studies using varied benchmarks and model families. The evaluation trend toward real-world datasets and executable validation strengthens the ecological validity of findings. The systematic identification of common weakness enumerations (CWEs) from the MITRE Top 25 list across many studies provides a reproducible basis for comparing vulnerability profiles Pearce et al. [2022], Aydın and Bahtiyar [2025]. The development of dedicated benchmarks such as CodeSecEval, SafeGenBench, and CWEval offers standardized platforms for future evaluations Wang et al. [2024], Li et al. [2025a], Peng et al. [2025]. Furthermore, the breadth of mitigation techniques—from prompt optimization and fine-tuning to reinforcement learning with security-aware rewards—demonstrates a maturing field with multiple avenues for improvement Nazzal et al. [2024], Islam et al. [2024], Melo [2025]. The use of evidence-card completeness and extraction audits, rather than formal quality scores, ensures transparency about the strengths and limitations of each included study.

5.3 Gaps and limitations in the literature

Despite progress, several gaps persist. Most studies focus on a narrow set of programming languages—predominantly Python, C/C++, and Java—leaving the generalizability to low-resource languages and legacy codebases underexplored Wang et al. [2025a], Yarema and Zagorodna [2026]. Real-world validation remains limited: many evaluations are conducted on synthetic or curated datasets, and the extent to which findings transfer to production systems is unclear Vaddiparthy [2025], Dora et al. [2025]. Non-determinism in LLM outputs is frequently overlooked, with fewer than a quarter of code

generation studies addressing this threat Ouyang et al. [2025]. The computational cost of state-of-the-art repair and alignment techniques (e.g., reinforcement learning with large models) poses a barrier to adoption Zhang [2025], Xu et al. [2024b]. While adversarial risks are well documented in controlled settings, their manifestation in real-world supply chain attacks and stealthy exploit generation is less understood Debar et al. [2024], Xu et al. [2024a]. Finally, current benchmarks often prioritize vulnerability identification over executable validation, limiting their ability to assess whether a fix preserves functionality Li et al. [2025a].

5.4 Implications for future research

Future research should prioritize the development of standardized, multi-lingual, repo-level benchmarks that include executable validation and cover a wider range of vulnerability types Peng et al. [2025], Li et al. [2025b]. Addressing non-determinism through rigorous reporting guidelines and experimental protocols is essential for reproducibility. Hybrid approaches that integrate LLMs with static analysis and fuzzing should be extended to larger-scale, real-world codebases to test scalability and practical utility Zhang [2025], Nunez et al. [2024]. Proactive security alignment—through fine-tuning on curated vulnerability scenarios or reinforcement learning with security-aware rewards—warrants further exploration, especially for low-resource domains such as hardware design and medical devices Xu et al. [2024b], Wang et al. [2025a], Ghosh et al. [2024]. The dual-use risks call for the development of ethical frameworks and red-teaming methodologies that are themselves systematically evaluated Wahréus et al. [2025], Albtosh and Aribowo [2025]. Finally, human-in-the-loop approaches that combine developer expertise with LLM-generated suggestions remain a promising but under-researched avenue for balancing productivity gains with security assurance Sankhe et al. [2025].

5.5 Limitations of the automated review process

The automated review process—from initial retrieval of 5,206 records to a top-ranked screening pool of 1,000 and eventual inclusion of 172 articles after LLM-assisted prioritization—introduces several limitations. First, the reliance on a single automated screening model (Qwen2.5-3B-Instruct) may have introduced selection bias, as articles ranked beyond the top 200 were not considered for full-text review. Second, the evidence-card completeness and extraction audit relied on metadata and available full-text for 192 of 200 article cards, meaning that 8 cards were based solely on abstracts, potentially limiting the depth of evidence captured. Third, the thematic synthesis was derived from an LLM-driven discovery process; while the eight themes were validated by the synthesis, alternative grouping or additional themes may have emerged from a fully manual analysis. Fourth, non-English studies and gray literature were excluded, and the review does not account for preprints or continuously updated models, which could affect the timeliness of the findings. Finally, because studies were permitted to belong to multiple themes, the counts per theme sum to more than the total number of included studies, a design choice that accurately reflects the multi-faceted nature of the evidence but complicates direct proportional comparisons. These limitations should be considered when interpreting the generalizability and completeness of the conclusions.

6 Conclusion

6.1 Overall conclusion

This systematic literature review synthesized evidence from 8 themes to examine how AI models impact the security of code generation. Across studies spanning vulnerability prevalence, detection and repair, mitigation techniques, adversarial risks, and domain-specific security, a consistent finding emerges: AI-generated code frequently introduces security vulnerabilities, with rates reaching up to approximately 40 % in some contexts Pearce et al. [2022], Kharma et al. [2026], Yan et al. [2025]. While models such as GPT-4 Turbo demonstrate high detection capabilities, they simultaneously exhibit poor classification accuracy and high false-positive rates Zhou et al. [2024], Gaspar Lopes et al. [2025]. Mitigation strategies—including fine-tuning, reinforcement learning, and prompt engineering—can reduce vulnerabilities, but their generalizability remains limited and the non-determinism of LLM outputs poses an under-addressed reliability threat Ouyang et al. [2025], Islam et al. [2024], Nazzal et al. [2024]. The dual-use nature of these models and the persistent risks across diverse domains underscore that secure AI-assisted coding is not solely a model capability issue but a systemic challenge requiring dedicated, multi-faceted solutions.

6.2 Most important evidence patterns

Several cross-study patterns and contrasts define the current landscape. First, vulnerability prevalence is systemic: studies consistently report that LLMs generate insecure code, with Common Weakness Enumerations (CWEs) from the MITRE Top 25 list appearing frequently Aydın and Bahtiyar [2025], Panina et al. [2024], Wang et al. [2024]. Second, a clear contrast exists between LLMs and traditional static analysis tools: LLMs achieve high detection rates (90–100 %) but with many false positives, whereas SAST tools have lower detection but fewer false alarms; combining both improves overall performance Zhou et al. [2024], Li et al. [2024]. Third, mitigation techniques show promise—CodingCare reduces reported CVEs by at least 50 % and SafeCodeRL reduces high-risk vulnerabilities by over 60 %—yet these gains are often confined to specific languages or narrow contexts Ding et al. [2025], Wang et al. [2026]. Fourth, adversarial risks and dual-use are well documented: GPT-4 achieves perfect scores in automated penetration tests while smaller models underperform, and prompt engineering significantly influences jailbreaking success Wahréus et al. [2025], Pratama et al. [2024], Xu et al. [2024a]. Finally, domain-specific studies across smart contracts, web applications, hardware RTL, and medical devices reveal that vulnerabilities in authentication and input validation persist even with fine-tuned models, and hardware designs exhibit particularly high risk Dora et al. [2025], Wang et al. [2025a], Collini et al. [2025]. These patterns collectively indicate that while LLMs offer productivity gains, their security implications demand proactive, hybrid, and context-aware approaches.

6.3 Practical and research implications

For practitioners, the findings imply that AI code assistants should not be trusted without human oversight or complementary security tooling. Integrating LLMs with static analysis, dynamic fuzzing, and syntax-aware repair—as demonstrated by frameworks achieving a 68.2 % vulnerability reduction while preserving 92.1 % functional integrity—represents a viable path forward Zhang [2025]. Developers should employ prompt rectification, domain-specific fine-tuning, and tiered model selection (escalating from lighter to advanced models based on vulnerability complexity) to balance speed and accuracy Hincu [2025]. For researchers, the review highlights urgent gaps: current benchmarks lack executable validation and real-world representativeness; evaluations often

overlook non-determinism and cross-language generalizability; and most studies focus on popular languages (Python, Java, C), leaving low-resource languages and smaller models underexplored. The field would benefit from standardized repo-level vulnerability benchmarks, metrics that capture both functionality and security, and systematic studies of emergent misalignment beyond code generation Betley et al. [2026], Li et al. [2025a], Peng et al. [2025].

6.4 Future work

Future research should address several open directions. First, comprehensive, multi-language, and context-rich benchmarks—such as CVEBench and CodeLMSec—should be extended to cover a wider range of vulnerability types and real-world codebases, with executable validation prioritized Wang et al. [2025b], Hajipour et al. [2024]. Second, the non-determinism of LLM outputs and its impact on security guarantees requires systematic investigation, as current evaluations rarely account for output variability Ouyang et al. [2025]. Third, adaptive and interpretable mitigation techniques that generalize across programming languages and industrial contexts are needed; reinforcement learning and preference learning approaches should be tested in diverse deployment settings Abdali et al. [2024], Islam et al. [2024]. Fourth, the dual-use risks—including supply chain attacks, model poisoning, and stealthiness of LLM-generated exploits—remain underexplored in real-world environments and demand ethical yet rigorous evaluation frameworks Wahréus et al. [2025], Hilario et al. [2024]. Finally, domain-specific studies should expand beyond current ecosystems (e.g., Solidity, C/C++, web frameworks) to emerging areas such as low-resource languages, embedded systems, and safety-critical domains, with human auditing integrated as a baseline Yu [2024], Collini et al. [2025], Ibrahim and Kashef [2025]. Addressing these gaps will be essential for realizing the productivity benefits of AI code generation without compromising software security.

References

Sara Abdali, Richard Anarfi, CJ Barberan, Jia He, and Erfan Shayegani. Securing large language models: Threats, vulnerabilities and responsible practices, 2024. URL https://arxiv.org/abs/2403.12503.

Luay Albtosh and Widi Aribowo. *Red Teaming and Stress-Testing Methodologies for LLM Security*, page 187–226. IGI Global Scientific Publishing, October 2025. ISBN 9798337371351. doi: 10.4018/979-8-3373-7133-7.ch006. URL http://dx.doi.org/10.4018/979-8-3373-7133-7.ch006.

Deniz Aydın and Şerif Bahtiyar. Security vulnerabilities in ai-generated javascript: A comparative study of large language models. In *2025 IEEE International Conference on Cyber Security and Resilience (CSR)*, page 200–205. IEEE, August 2025. doi: 10.1109/csr64739.2025.11130176. URL http://dx.doi.org/10.1109/csr64739.2025.11130176.

Jan Betley, Niels Warncke, Anna Sztyber-Betley, Daniel Tan, Xuchan Bao, Martín Soto, Megha Srivastava, Nathan Labenz, and Owain Evans. Training large language models on narrow tasks can lead to broad misalignment. *Nature*, 649(8097):584–589, January 2026. ISSN 1476-4687. doi: 10.1038/s41586-025-09937-5. URL http://dx.doi.org/10.1038/s41586-025-09937-5.

Luca Collini, Baleegh Ahmad, Joey Ah-kiow, and Ramesh Karri. Marvel: Multi-agent rtl vulnerability extraction using large language models, 2025. URL https://arxiv.org/abs/2505.11963.

Herve Debar, Sven Dietrich, Pavel Laskov, Emil C. Lupu, and Eirini Ntoutsi. Emerging security challenges of large language models, 2024. URL https://arxiv.org/abs/2412.17614.

Zhiguo Ding, Songyang Wu, Yilin Zhang, Tao Yang, Yingna Li, Zhuhua Li, Hang Shao, and Yicong Shi. Codingcare: Ai code generation security framework for common vulnerability mitigation. In *Proceedings of the 2025 4th International Conference on Intelligent Systems, Communications and Computer Networks*, ISCCN 2025, page 307–312. ACM, February 2025. doi: 10.1145/3732945.3732990. URL http://dx.doi.org/10.1145/3732945.3732990.

Swaroop Dora, Deven Lunkad, Naziya Aslam, S. Venkatesan, and Sandeep Kumar Shukla. The hidden risks of llm-generated web application code: A security-centric evaluation of code generation capabilities in large language models, 2025. URL https://arxiv.org/abs/2504.20612.

Diogo Gaspar Lopes, Tiago Espinha Gasiba, Sathwik Amburi, and Maria Pinto-Albuquerque. Can open large language models catch vulnerabilities? volume 133, pages 4:1–4:14. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025. doi: 10.4230/OASICS.ICPEC.2025.4. URL https://drops.dagstuhl.de/entities/document/10.4230/OASIcs.ICPEC.2025.4.

Rikhiya Ghosh, Oladimeji Farri, Hans-Martin von Stockhausen, Martin Schmitt, and George Marica Vasile. Cve-llm : Automatic vulnerability evaluation in medical device industry using large language models, 2024. URL https://arxiv.org/abs/2407.14640.

Hossein Hajipour, Keno Hassler, Thorsten Holz, Lea Schönherr, and Mario Fritz. Codelmsec benchmark: Systematically evaluating and finding security vulnerabilities in black-box code language models. In *2024 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, page 684–709. IEEE, April 2024. doi: 10.1109/satml59370.2024.00040. URL http://dx.doi.org/10.1109/satml59370.2024.00040.

Eric Hilario, Sami Azam, Jawahar Sundaram, Khwaja Imran Mohammed, and Bharanidharan Shanmugam. Generative ai for pentesting: the good, the bad, the ugly. *International Journal of Information Security*, 23(3):2075–2097, March 2024. ISSN 1615-5270. doi: 10.1007/s10207-024-00835-x. URL http://dx.doi.org/10.1007/s10207-024-00835-x.

Veronica Hincu. Integration of ai algorithms in the vulnerability testing process. In *Creating the Society of Consciousness, TELE-2025*, page 85–88. Academy of Economic Studies, July 2025. doi: 10.53486/csc2025.19. URL http://dx.doi.org/10.53486/csc2025.19.

S M Mostaq Hossain, Amani Altarawneh, and Jesse Roberts. Leveraging large language models and machine learning for smart contract vulnerability detection. In *2025 IEEE 15th Annual Computing and Communication Workshop and Conference (CCWC)*, page 00577–00583. IEEE, January 2025. doi: 10.1109/ccwc62904.2025.10903869. URL http://dx.doi.org/10.1109/ccwc62904.2025.10903869.

Nourhan Ibrahim and Rasha Kashef. Exploring the emerging role of large language models in smart grid cybersecurity: a survey of attacks, detection mechanisms, and mitigation strategies. *Frontiers in Energy Research*, 13, March 2025. ISSN 2296-598X. doi: 10.3389/fenrg.2025.1531655. URL http://dx.doi.org/10.3389/fenrg.2025.1531655.

Nafis Tanveer Islam, Mohammad Bahrami Karkevandi, and Peyman Najafirad. Code security vulnerability repair using reinforcement learning with large language models, 2024. URL https://arxiv.org/abs/2401.07031.

Mohammed F. Kharma, Soohyeon Choi, Mohammad Alkhanafseh, and David Mohaisen. Security and quality in llm-generated code: A multi-language, multi-model analysis. *IEEE Transactions on Dependable and Secure Computing*, 23(3):7300–7314, May 2026. ISSN 2160-9209. doi: 10.1109/tdsc.2026.3672745. URL http://dx.doi.org/10.1109/tdsc.2026.3672745.

Xinghang Li, Jingzhe Ding, Chao Peng, Bing Zhao, Xiang Gao, Hongwan Gao, and Xinchen Gu. Safegenbench: A benchmark framework for security vulnerability detection in llm-generated code, 2025a. URL https://arxiv.org/abs/2506.05692.

Yue Li, Xiao Li, Hao Wu, Minghui Xu, Yue Zhang, Xiuzhen Cheng, Fengyuan Xu, and Sheng Zhong. Everything you wanted to know about llm-based vulnerability detection but were afraid to ask, 2025b. URL https://arxiv.org/abs/2504.13474.

Ziyang Li, Saikat Dutta, and Mayur Naik. Iris: Llm-assisted static analysis for detecting security vulnerabilities, 2024. URL https://arxiv.org/abs/2405.17238.

Rui Melo. Securing language models against vulnerability encoding. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, FSE Companion '25, page 1277–1278. ACM, June 2025. doi: 10.1145/3696630.3731466. URL http://dx.doi.org/10.1145/3696630.3731466.

Mahmoud Nazzal, Issa Khalil, Abdallah Khreishah, and NhatHai Phan. Promsec: Prompt optimization for secure generation of functional source code with large language models (llms). In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, page 2266–2280. ACM, December 2024. doi: 10.1145/3658644.3690298. URL http://dx.doi.org/10.1145/3658644.3690298.

Claudia Negri-Ribalta, Rémi Geraud-Stewart, Anastasia Sergeeva, and Gabriele Lenzini. A systematic literature review on the impact of ai models on the security of code generation. *Frontiers in Big Data*, 7, May 2024. ISSN 2624-909X. doi: 10.3389/fdata.2024.1386720. URL http://dx.doi.org/10.3389/fdata.2024.1386720.

Ana Nunez, Nafis Tanveer Islam, Sumit Kumar Jha, and Peyman Najafirad. Autosafecoder: A multi-agent framework for securing llm code generation through static analysis and fuzz testing, 2024. URL https://arxiv.org/abs/2409.10737.

Shuyin Ouyang, Jie M. Zhang, Mark Harman, and Meng Wang. An empirical study of the non-determinism of chatgpt in code generation. *ACM Transactions on Software Engineering and Methodology*, 34(2):1–28, January 2025. ISSN 1557-7392. doi: 10.1145/3697010. URL http://dx.doi.org/10.1145/3697010.

O.A. Panina, D.A. Yurin, and I.Yu. Sykhov. Security risks of using large language models (llm) in systems. *Informatization and communication*, 6, 2024. ISSN 2078-8320. doi: 10.34219/2078-8320-2024-15-6-65-72. URL http://dx.doi.org/10.34219/2078-8320-2024-15-6-65-72.

Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. Asleep at the keyboard? assessing the security of github copilot's code contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*, page 754–768. IEEE, May 2022. doi: 10.1109/sp46214.2022.9833571. URL http://dx.doi.org/10.1109/sp46214.2022.9833571.

Jinjun Peng, Leyi Cui, Kele Huang, Junfeng Yang, and Baishakhi Ray. Cweval: Outcome-driven evaluation on functionality and security of llm code generation. In *2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*, page 33–40. IEEE, May 2025. doi: 10.1109/llm4code66737.2025.00009. URL http://dx.doi.org/10.1109/llm4code66737.2025.00009.

Derry Pratama, Naufal Suryanto, Andro Aprila Adiputra, Thi-Thu-Huong Le, Ahmada Yusril Kadiptya, Muhammad Iqbal, and Howon Kim. Cipher: Cybersecurity intelligent penetration-testing helper for ethical researcher. *Sensors*, 24(21):6878, October 2024. ISSN 1424-8220. doi: 10.3390/s24216878. URL http://dx.doi.org/10.3390/s24216878.

Purvi Sankhe, Neeta Patil, Minakshi Ghorpade, Pratibha Prasad, and Monisha Linkesh. Empirical analysis of ai-assisted code generation tools impact on code quality, security and developer productivity. *International Journal For Multidisciplinary Research*, 7(6), November 2025. ISSN 2582-2160. doi: 10.36948/ijfmr.2025.v07i06.61350. URL http://dx.doi.org/10.36948/ijfmr.2025.v07i06.61350.

Harshith Vaddiparthy. Real-time ai code security auditing: Automated vulnerability detection and remediation through meta-experimental analysis. September 2025. doi: 10.21203/rs.3.rs-7529613/v1. URL http://dx.doi.org/10.21203/rs.3.rs-7529613/v1.

Dang H. Vu, Lam D. Dao, Anh M. T. Bui, Phuong T. Nguyen, Davide Di Ruscio, and Massimiliano Di Penta. On the gap between declarative vulnerability knowledge and secure code generation in large language models. 2026. doi: 10.2139/ssrn.6779013. URL http://dx.doi.org/10.2139/ssrn.6779013.

Johan Wahréus, Ahmed Mohamed Hussain, and Panos Papadimitratos. Cysecbench: Generative ai-based cybersecurity-focused prompt dataset for benchmarking large language models, 2025. URL https://arxiv.org/abs/2501.01335.

Jiexin Wang, Xitong Luo, Liuwen Cao, Hongkui He, Hailin Huang, Jiayuan Xie, Adam Jatowt, and Yi Cai. Is your ai-generated code really safe? evaluating large language models on secure code generation with codeseceval, 2024. URL https://arxiv.org/abs/2407.02395.

Lingxiang Wang, Hainan Zhang, Qinnan Zhang, Ziwei Wang, Hongwei Zheng, Jin Dong, and Zhiming Zheng. Codebc: A more secure large language model for smart contract code generation in blockchain, 2025a. URL https://arxiv.org/abs/2504.21043.

Peiran Wang, Xiaogeng Liu, and Chaowei Xiao. Cve-bench: Benchmarking llm-based software engineering agent's ability to repair real-world cve vulnerabilities. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, page 4207–4224. Association for Computational Linguistics, 2025b. doi: 10.18653/v1/2025.naacl-long.212. URL http://dx.doi.org/10.18653/v1/2025.naacl-long.212.

Zhihua Wang, Junfan Chen, Zixiang Wei, Lan Lin, and Guoxiang Tong. Safecoderl: Security-constrained multi-agent reinforcement learning for trustworthy llm-generated iot/cps software. *Sensors*, 26(11):3502, June 2026. ISSN 1424-8220. doi: 10.3390/s26113502. URL http://dx.doi.org/10.3390/s26113502.

Jiacen Xu, Jack W. Stokes, Geoff McDonald, Xuesong Bai, David Marshall, Siyue Wang, Adith Swaminathan, and Zhou Li. Autoattacker: A large language model guided system to implement automatic cyber-attacks, 2024a. URL https://arxiv.org/abs/2403.01038.

Xiangzhe Xu, Zian Su, Jinyao Guo, Kaiyuan Zhang, Zhenting Wang, and Xiangyu Zhang. Prosec: Fortifying code llms with proactive security alignment, 2024b. URL https://arxiv.org/abs/2411.12882.

Hao Yan, Swapneel Suhas Vaidya, Xiaokuan Zhang, and Ziyu Yao. Guiding ai to fix its own flaws: An empirical study on llm-driven secure code generation, 2025. URL https://arxiv.org/abs/2506.23034.

Oleh Yarema and Nataliya Zagorodna. Comparative analysis of the effectiveness of software code vulnerability detection using llm and sast. *Cybersecurity Education Science Technique*, page 878, March 2026. ISSN 2663-4023. doi: 10.28925/2663-4023.2026.32.1165. URL http://dx.doi.org/10.28925/2663-4023.2026.32.1165.

Jeffy Yu. Retrieval augmented generation integrated large language models in smart contract vulnerability detection, 2024. URL https://arxiv.org/abs/2407.14838.

Beilei Zhang. Code generation security in llms: A hybrid detection and post-processing framework for vulnerability mitigation. In *Poster Volume The 2025 Twenty-First International Conference on Intelligent Computing July 26-29, 2025 Ningbo, China*, ICIC, page 833–845. The Society of International Computing, 2025. doi: 10.65286/icic.v21i4.69146. URL http://dx.doi.org/10.65286/icic.v21i4.69146.

Xin Zhou, Duc-Manh Tran, Thanh Le-Cong, Ting Zhang, Ivana Clairine Irsan, Joshua Sumarlin, Bach Le, and David Lo. Comparison of static application security testing tools and large language models for repo-level vulnerability detection, 2024. URL https://arxiv.org/abs/2407.16235.